COMP3600 LAB2

🧩 AI Application – Pathfinding in a Maze

In this lab session, we will explore our first AI application: **finding a path through a maze**. To accomplish this, we will apply either **Depth-First Search (DFS) or Breadth-First Search (BFS)**, both of which were covered in this week's lessons. The maze will be **generated using the Pyamaze Python Library**.

To better understand BFS and DFS, you may wish to **visualize these algorithms in action** using interactive tools before proceeding with the implementation at the below link:

🔗 Graph Traversal Visualization

Exercise 1: Generating a Maze and Using Pyamaze's Built-in Path Search

1. Use the **Pyamaze** library to generate a **default maze**.
2. Execute the built-in pathfinding algorithm using the `tracePath()` function.

Code Example:

```
from pyamaze import maze

m = maze()
m.CreateMaze()
# Run the built-in pathfinding visualization
m.run()
```

Exercise 2: Implementing DFS for Maze Solving

1. Implement a custom function called `dfs()`. The function should take the maze object `m` as an input argument and return a **list of cell coordinates** representing the search path taken by the DFS algorithm.
2. Create an agent `dfs_agent1` that **visualizes the search space** by filling each visited cell with **blue color**.
3. Experiment with different **maze sizes** (e.g., `maze(15,15)`) to observe DFS behavior in different environments.

Exercise 3: Extracting the Optimal Path and Performance Evaluation

1. Implement another agent, `dfs_agent2`, which **visualizes only the optimal path** from the start to the goal, leaving a footprint of the **actual search path**.
2. Measure and display the algorithm's performance, including:
   - **Execution time**
   - **Number of nodes visited**
3. Display the execution time and the number of visited nodes in the output.

Bonus Challenge:

- Implement **recursive vs. iterative DFS**, then compare their execution time and efficiency.
- Modify the visualization to **show live backtracking** (e.g., mark backtracked paths with a fading color).

This structured approach will help you understand **how AI algorithms navigate and solve real-world problems** such as maze traversal.

Happy coding! 🚀

Further Exploration

Want to see real-world applications of **DFS and BFS** algorithms? Check out this collection of graph traversal problems:

🔗 Graph Problems on LeetCode

✨ Done with Passion

Sultan Qaboos University - COMP3600

Dr. Noushath Shaffi

TA: Malak Al Bahri -137615-